Architetture dei Sistemi Di Elaborazione **Laboratory 8**	Delivery date: 05 December 2025 Expected delivery of lab_08.zip must include: - zipped project folders for Exercise1, Exercise2 - this lab track completed and converted to pdf format.

Exercise 1) Experiment the SVC instruction.

Download the template project for Keil µVision "*ABI_C+ASM*" from the course material.

Write, compile, and execute a code that invokes an SVC instruction through inline assembly in the C main of your application.

By means of invoking a **SuperVisor Call**, implement an error-correction method to enable a fault-tolerant data transmission by leveraging Hamming Code (7,4). This code adds redundancy bits (parity bits) to the data to allow the receiver to detect and correct errors. Instead of simply repeating bits, we calculate specific parity bits based on the data bits.

For example, suppose that we send the 4-bit message «1010» (d1d2d3d4). It will be encoded into a 7-bit sequence by adding 3 parity bits (p1p2p3) at specific positions (indices 0, 1, and 3), resulting in the sequence «1011010».

1. Correct Transmission Received Message:

   1011010

   Decoded Data:

   1010

2. Faulty Transmission Received Message:

   1010010 (Bit 4 flipped)

   Decoded Data

   1010

It can correct a message when one of the transmitted bits is flipped - a single bit error - meaning that the correcting ability of this code is 1 bit.

You are required to implement the following ASM code.

Use a SuperVisor Call (SVC) to request encoding of a binary message. The message to be encoded is stored in the first 8 bits of the SVC instruction (bit 0 through 7). Only consider the lower 4 bits as valid data (d1 through d4).

In SVC handler, implement the Hamming Code (7,4) on the input message.

For the 4-bit input (d1, d2, d3, d4), calculate 3 parity bits (p1, p2, p3) using XOR operations:

p1	*d1* ***XOR*** *d2* ***XOR*** *d4*
p2	*d1* ***XOR*** *d3* ***XOR*** *d4*
p3	*d2* ***XOR*** *d3* ***XOR*** *d4*

Construct the 7-bit result using standard Hamming positions: p1 p2 d1 p3 d2 d3 d4

Return the encoded message using the stack. Save the encoded message in R0.

Example:

SVC 0xA; 2_1010 binary value of the SVC number (d1=1, d2=0, d3=1, d4=0)

Returning from the SVC, R0 will contain the value “1011010”.

Q1: Describe how the stack structure is used by your project.
Lo stack usato è `PSP` in quando le funzioni chiamate dal programma di startup modificano il bit di controllo in modo che venga usato in modalità `Unprivileged`.
Inoltre, lo stack viene anche utilizzato per salvare il valore `R0` (risultato hamming). `SVC_Handler` viene chiamato dal file `ASM_funct.s` e, al ritorno, in `R0` vi è il valore correttamente calcolato da `SVC`.

Exercise 2) Inverse Square Root, the Quake III Fast Approximation

- Download the template project for Keil µVision “*ABI_C+ASM*” from the course material.

In computer graphics, lighting and reflection calculations require normalizing vectors billions of times per second. To normalize a vector, you must divide each component by the vector's length (the square root of the sum of squares). This requires computing:

$$f(x) = 1 / sqrt(x)$$

In the late 1990s, the video game Quake III Arena revolutionized 3D gaming. Its source code contained a legendary function for calculating this value up to 4x faster than the hardware could. The code is famous for its cryptic constant "0x5f3759df" and the profane comment "what the f***?" left by the original programmer next to the bit-shifting line. While often attributed to John Carmack, the algorithm's roots trace back to Ardent Computer in the mid-80s.

The full algorithm explanation can be found here:

https://www.youtube.com/watch?v=p8u_k2LIZyo

The algorithm relies on the fact that the bit pattern of a floating-point number, when interpreted as an integer, approximates its logarithm.

The method consists of two main steps:

1. An initial guess generated by treating the floating-point bits as an integer, shifting them right by one, and subtracting them from a "magic constant."
2. One iteration of Newton's method to refine the precision.

   The approximation step (The Magic):

   $$i = 0x5f3759df - (i >> 1) \text{ (Eq. 1)}$$

   Where 'i' is the input float interpreted as a long integer.

   The Newton-Raphson refinement step:

   $$y = y * (1.5 - (x2 * y * y)) \text{ (Eq. 2)}$$

Where 'y' is the current guess, and 'x2' is half the original input.

Mathematical operations in computers are finite! The Arm Cortex-M3 does not have a Floating Point Unit (FPU). **You will perform the "Magic" bit-hack in Assembly** (since it treats floats as integers anyway) and the Newton refinement in C.

You are required to implement the solution using a mix of C and Assembly.

Declare the input values (floating point numbers) into a read-only area of data 4-byte aligned (into an assembly file) as follows. These values represent squared distances that need to be inverted.

Note that these are Hex representations of IEEE-754 floats.

Input_Values	DCD	0x40000000, 0x40800000, 0x41200000, 0x41C80000
	DCD	0x42C80000, 0x447A0000, 0x3F800000, 0x42480000
NUM_VALUES	DCB	8

The parsing of Input_Values must be done in C. Remember that the extern keyword must be used for referencing assembly data structures:

extern int _Input_Values;

extern char _NUM_VALUES;

Hint: be careful on how you load data from the vectors or variable you defined in the assembly section.

extern unsigned int *_Input_Values*; //Gives you the **value** of the first element of Input_Values

volatile unsigned int * vector=&_ *Input_Values*; //Gives you the **address** of the first element and so the beginning of the vector. This is how you obtain the pointer of the matrix from the assembly data section.

In the loop body of your C code, you must iterate through the values and calculate the initial guess. This calculation must be done by calling an assembly function with the following prototype:

int fast_magic_calc(int input_float_as_int)

This function implements Eq. 1:

1. Load the magic constant 0x5f3759df.
2. Perform the logical shift right (LSR) by 1 on the input.
3. Subtract the shifted input from the magic constant.
4. Return the result (which C will cast back to a float variable).

After getting this integer from Assembly, your C code should cast it back to a float and perform one iteration of Newton's method (Eq. 2) to refine the result.

To verify the accuracy of your fast approximation, you must compare it to the standard calculation. You are required to compute the "Standard" Inverse Square Root:

Standard = 1.0 / sqrt(input) // HINT: include <math.h>

Goal:

1. Iterate through the Input_Values.
2. Call fast_magic_calc (ASM) to get the approximation.
3. Refine the result in C (Newton step).
4. Compute the standard value using 1 / sqrt(<val>) from math.h.
5. Store the difference (Error) between the Fast method and the Standard method in a dedicated vector in C (float ERRORS[_NUM_VALUES]).
6. Insert the value in ERRORS in the following section

Q1: Value in ERRORS:
`0.000176727772, 0.000846415758, 0.000541985035, 0.000310242176, 0.000155121088, 5.29363751e-05, 0.00169283152, 6.54160976e-05`